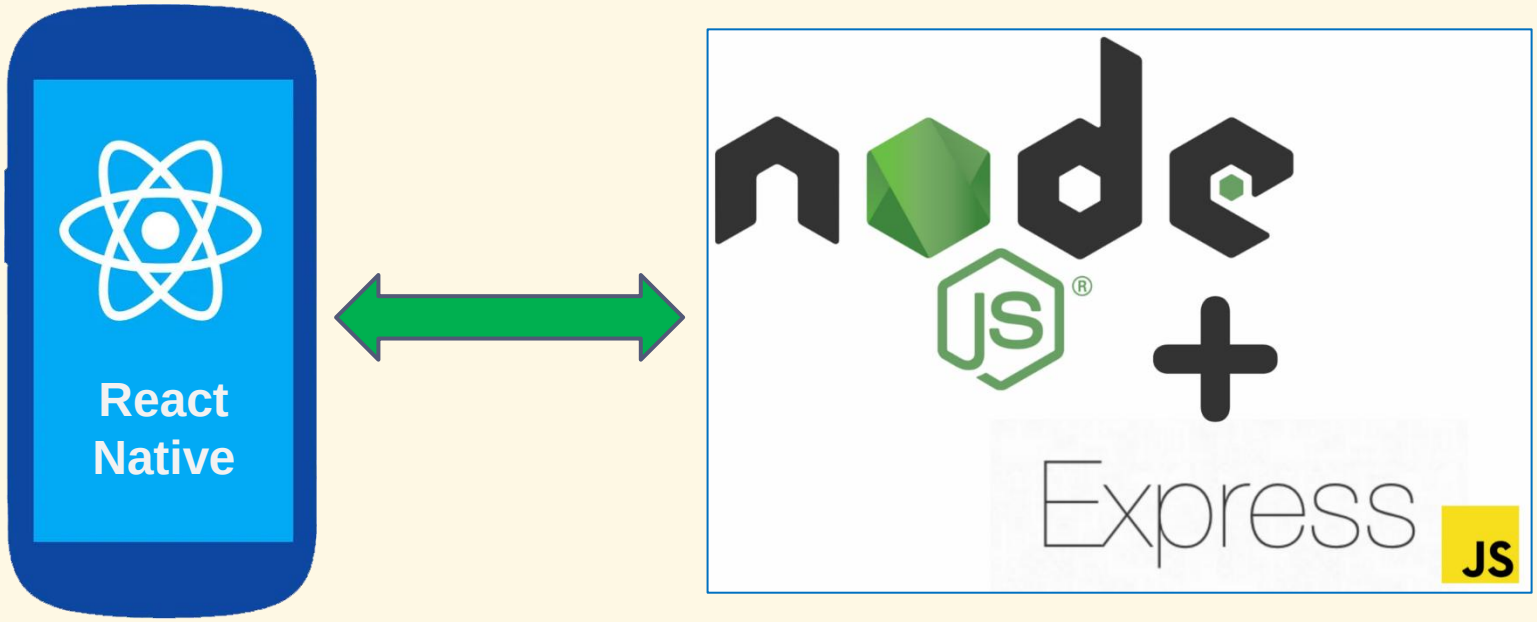
Networking

Communication with Server

Introduction

- Many mobile apps need to communicate with a server
- Most common methods:
  - Making a HTTP GET/POST request to a REST API
  - Using WebSocket



Client

Server

REST (Representational State Transfer) API

- A set of functions used by clients to make requests to a server and receive responses through HTTP
- Allows to create, update, delete data managed by the server
- Advantages:
  - Simple and lightweight
  - Secure (with HTTPS)
  - Not blocked by firewalls
  - Flexible and scalable: clients and servers are coupled very loosely
- Libraries to make REST API calls in RN:
  - Axios: https://www.npmjs.com/package/axios
  - Fetch API: https://www.npmjs.com/package/node-fetch

Server Side: GET API Endpoint with Express

```
const express = require('express');
const app = express();

const port = 3001;

// CORS (Cross-Origin Resource Sharing) control
app.use((req, res, next) => {
  res.append('Access-Control-Allow-Origin', ['*']);
  res.append('Access-Control-Allow-Methods', 'DELETE,GET,PATCH,POST,PUT');
  res.append('Access-Control-Allow-Headers', 'Content-Type,Authorization');

  if (res.method == 'OPTIONS') res.send(200);
  else next();
});

app.get('/api/add', (req, res) => {
  const x = +req.query.x;
  const y = +req.query.y;
  res.json({result: x + y});
});

app.listen(port, () => `Server running on port ${port}`);
```

Client Side: GET API Request from RN

```
const params = new URLSearchParams({
  x: 10, y: 20
});

fetch(`http://${serverUrl}/api/add?` + params)
.then(res => res.json())
.then(data => {
  // do something with data.result
})
.catch(err => {
  // ...
});
```

Make a POST API Request

- Server
  - ```
    const bodyParser = require('body-parser');
    app.use(bodyParser.json());

    app.post('/api/add', (req, res) => {
      const x = +req.body.x;
      const y = +req.body.y;
      res.json({result: x + y});
    });
    ```

- Client
  - ```
    const params = { x, y };

    fetch(`http://${serverUrl}/api/add`, {
      method: 'POST',
      headers: {
        'Content-Type': 'application/json'
      },
      body: JSON.stringify(params)
    })
    // ...
    ```

Exercises

- Create an app allowing the user to enter two numbers and calculate the summation through a GET API.
  - Modify the app to use a POST API.
- Create an app showing the weather forecast with two screens:
  - Home: showing a list of cities returned from `/api/cities`.
  - Details: showing the detailed weather forecast information for a city returned from `/api/weather?id=${cityId}` when the user taps on a city from the Home screen.

Final Notes

- Use HTTPS instead of HTTP for security
- Since mobile apps and APIs usually undergo a lot of updates, they should have a robust process for version control to manage the changes better
- The APIs for mobile apps should ensure all functionalities accessed by the user while being offline, to be reconciled when going online

Setting Up a HTTPS Server with ExpressJS

- Code

```
const express = require('express');
const https = require('https');
const fs = require('fs');

const PORT = 8443;

const credentials = {
    key: fs.readFileSync('private.key', 'utf8'),
    cert: fs.readFileSync('public.crt', 'utf8')
};

const app = express();
const server = https.createServer(credentials, app);

server.listen(PORT, () =>
    console.log('Server is running...'));
```

- Generation of a self-signed certificate

```
openssl req -x509 -nodes -days 3650 -newkey rsa:2048
-keyout private.key -out public.crt
```

User Authentication

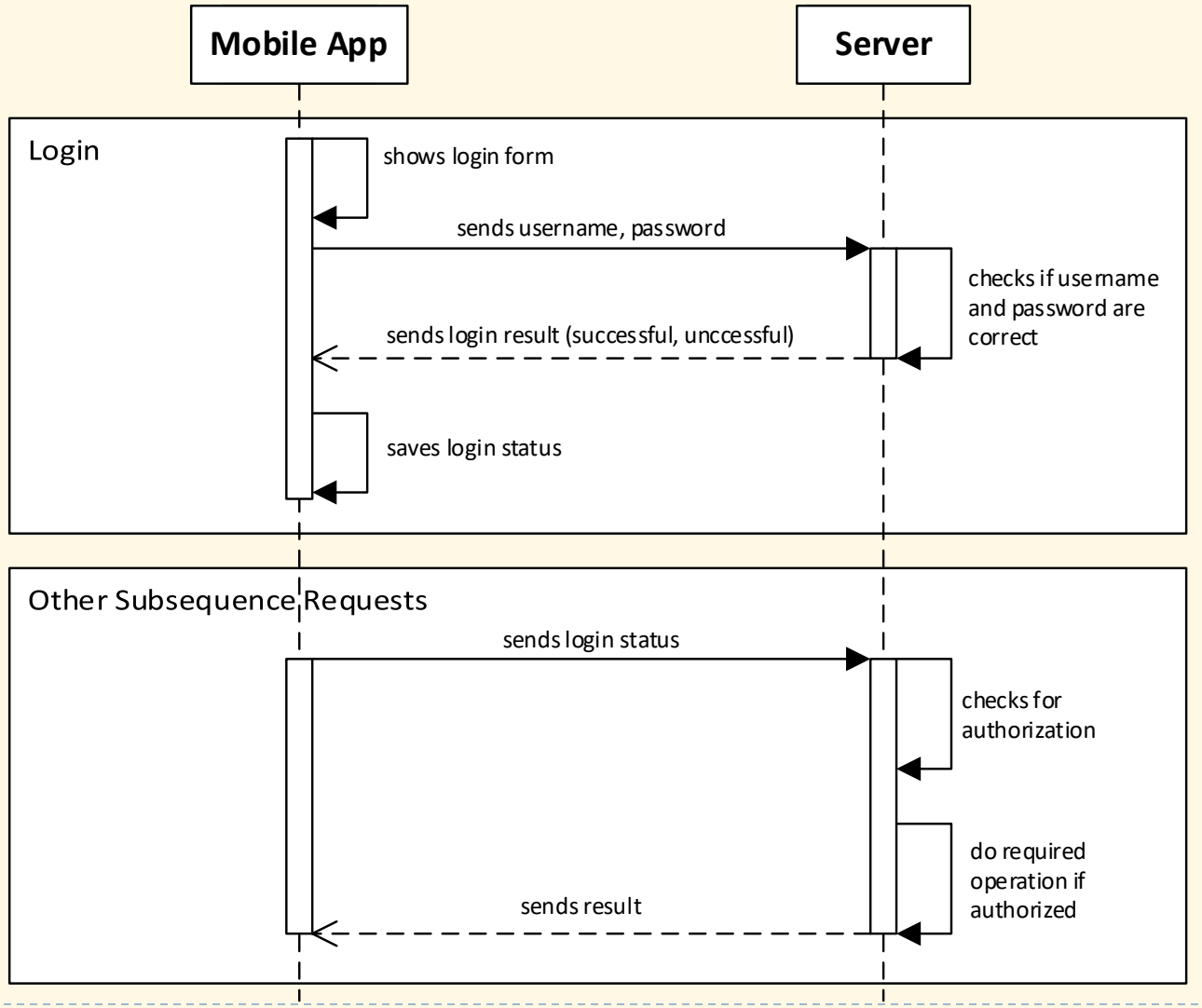
Overview

- Many mobile apps require users to authenticate
- The most basic method for authentication is using a pair of username and password
- Operations:

User (on mobile app)	Server
Signs up	Validates information and create account
Signs in	Checks username/password and registers session
Signs out	Unregisters session
Others	Checks for session and proceeds if authorized, or returns error otherwise

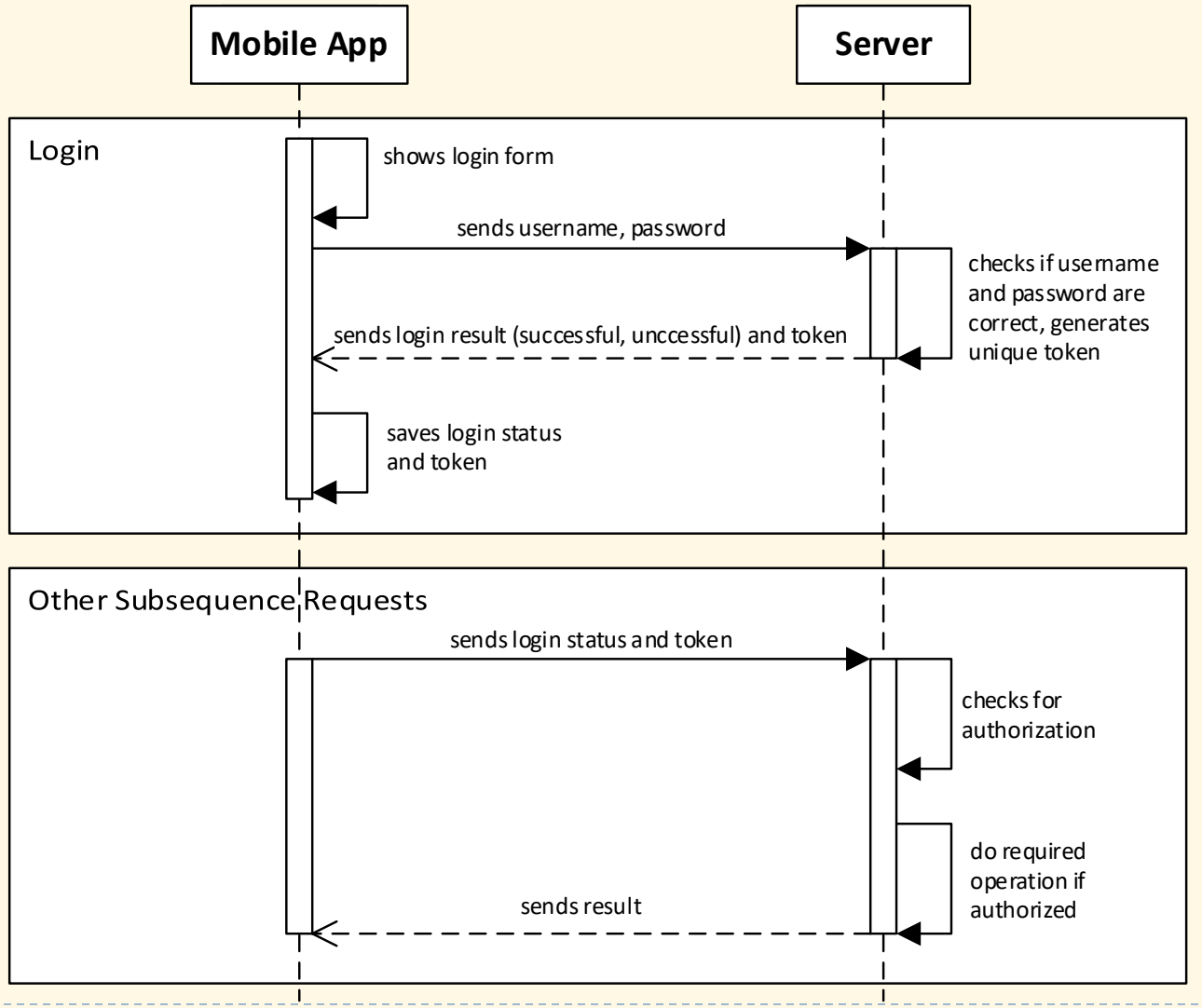
- Big question: how to securely manage the user session?

Is the Following Sign-in Process Safe?



- Register session = send login status
- Unregister session = reset login status
- Check session validity = check login status

Token-based Session Management



- Register session = generate unique token
- Unregister session = remove token
- Check session validity = check if token exists

Exercise

- Create a simple app protected by a login screen.

JSON Web Token (JWT)

- An open standard for secure communication between client and server
  - Token-based authentication
  - Information and token are digitally signed
  - Payload is a JSON object

- JavaScript library for JWT:
  - https://www.npmjs.com/package/jsonwebtoken
  - Install (for server):
    - `npm install jsonwebtoken`
    - `const jwt = require('jsonwebtoken');`

Server: Authentication Example

```
const userInfo = checkLogin(username, password);
if (!userInfo) return res.json({ success: false });

const data = {
  id: userInfo.id
};
const options = {
  expiresIn: '30m'   // 30 minutes
};
const token = jwt.sign(data, SECRET_STRING, options);
return res.json({ success: true, token });
```

Server: Check Authorization Example

```
// Middleware to be applied on any route that requires to
// check for authorization
function checkAuthorization(req, res, next) {
  // Token may be sent either through headers,
  // POST body or query string
  const token = req.get('Authorization') ||
                req.body.token || req.query.token;
  if (!token) return res.json({ success: false });

  jwt.verify(token, SECRET_STRING, (err, data) => {
    if (err) return res.json({ success: false });
    const userInfo = userInfoById(data.id);
    if (!userInfo) return res.json({ success: false });

    req.userInfo = userInfo;
    next();
  });
}
```

Final Notes

- How to securely store the token on the mobile device for long-term automatically login?
  - Use SecureStore library:
    - https://www.npmjs.com/package/expo-secure-store

- Further references
  - OAuth 2.0
  - Third-party authentication providers: Google, Facebook,…